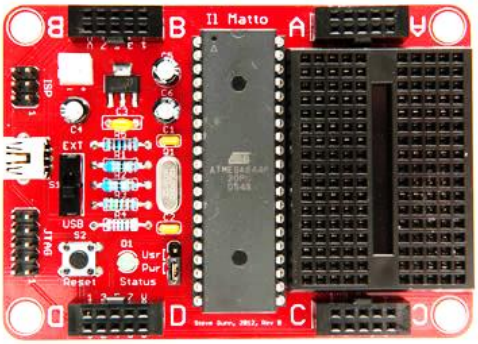
University of Southampton

ELEC Laboratory Programme

Laboratory manual

ρ4

The Cyber-Physical Bridge



Again? Yes. Again.

In this lab we will be controlling the Il-Matto to implement a simple Digital-Analogue Conversion. However, there's a catch! The Il Matto will be taking in commands from a hosted program and implementing it in a physical system.

Revision History

October 25, 2025	Dr Ivan Ling	New for 2025/26
November 4, 2026	Dr Ivan Ling	Edited deliverables. Clearer expectations

Electronics and Computer Science

© Electronics and Computer Science, University of Southampton

Laboratory Overview and Information

Schedule	
Preparation time:	3 hour
Lab time:	3 hours
Items provided	
Tools:	None
Components:	Resistors and capacitors
Equipment:	Oscilloscope
Software:	AVR-GCC / TurboAVR, PuTTY (or any serial terminal app)
Items to bring	
Essentials:	A full list is available on the Laboratory website at https://secure.ecs.soton.ac.uk/notes/ellabs/databook/essentials/

Initial Instructions

Before you come to the lab, it is essential that you read through this document and complete ***all*** of the preparation work in section 2. Only preparation which is recorded in your laboratory logbook will contribute towards your mark for this exercise. There is no objection to several students working together on preparation, as long as all understand the results of that work. Before starting your preparation, read through all sections of these notes so that you are fully aware of what you will have to do in the lab.

Academic Integrity

If you undertake the preparation jointly with other students, it is important that you acknowledge this fact in your logbook. Similarly, you may want to use sources from the internet or books to help answer some of the questions. Again, record any sources in your logbook.

© Electronics and Computer Science, University of Southampton

1 Aims, Learning Outcomes and Outline

This laboratory exercise aims to:

- Introduce the fundamentals of cyber-physical systems.
- Introduce object-oriented hardware abstraction: designing a small class library with inheritance and virtual functions on a resource-constrained target.
- Familiarise yourself with serial communication on embedded C/C++.

Having successfully completed the lab, you will be able to:

- Design and implement a hardware abstraction layer (HAL) as a C++ class hierarchy
- Design and implement an embedded system that is controlled by a PC through serial communication.
- Use a microcontroller to continuously monitor an analogue signal and respond with appropriate output in real time.

For this exercise, you will build a digital to analogue converter (DAC) that takes in an 8-bit number and generates a corresponding analogue voltage level. In other words, it is the complete opposite of the ADC. To do this, you will first study the behaviour of a passive low-pass filter (LPF) with a circuit simulation. You will then build, using components you have left over from Part 1 programming labs (and maybe a few more extra components from the lab), to build a working LPF and connect it with the Il Matto's PWM output to form a simple DAC.

Before any control code is written, you will design and build a small hardware abstraction library of your own. Every peripheral you use in this lab (PWM output, ADC input, LED…) must be driven through your own classes. Then, you will implement a simple control algorithm in your code to keep the output voltage at your setpoint, regardless of external disturbances. Finally, you may choose to add any additional features according to your ability.

You will drive your Il Matto directly from a serial terminal such as PuTTY, nothing fancy please. Building a proper graphical user interface is exactly what ρ5 is about, so any “fancy” interface you crate now will not be useful in the next lab.

Lab Partners 👫
For this assignment, you should complete the work with your project partner. However, it is important to understand that all students will be assessed individually.

Since you work in a pair, we highly recommend a classic divide-and-conquer strategy. This is a practical opportunity to apply the team-based software development methods covered in lectures, so don't be a lone wolf!

© Electronics and Computer Science, University of Southampton

2 Preparation

Read through the course handbook statement on safety and safe working practices, and your copy of the standard operating procedure. Make sure that you understand how to work safely. Read through this document so you are aware of what you will be expected to do in the lab. This project builds on the various skills you have acquired from C1 to C10. You will build a computer-controlled DAC based on a simple passive LPF circuit as shown in the figure below.

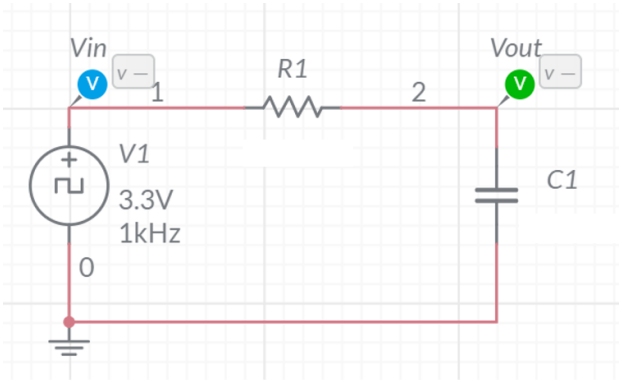


Figure 1: Circuit Schematic

You should start by making sure that all the required Software packages are installed.

2.1 Simulation

With a circuit simulation tool of your choice, create a model of the circuit shown in Figure 1. Work out the range of values available for PWM and select a sensible resistor and capacitor value. Record your observation and your selected RC values along with your justifications.

2.2 Read and understand PID control systems

Read up on PID control and try to understand how to implement them in C. Make notes of the equations involved in your logbook.

Write a draft of the equation in in your logbook (both mathematical and in C++ syntax) .

2.3 Design your Hardware Abstraction Layer

In your logbook, sketch a class diagram for a minimal HAL. You need at least: an **abstract base class that defines a common interface**, and concrete classes for a **Serial, PWM, and ADC** that inherit from it. Decide: what operations are common to all peripherals? What belongs only to an output, or only to an input?

2.4 Il Matto – Health check

Prepare your Il Matto for the lab. Make sure your Il Matto is still working. If it is not working, please check and fix any pending issues.

© Electronics and Computer Science, University of Southampton

3 Laboratory Work

3.1 Build your Hardware Abstraction

Implement the classes designed in prep using the register-level code from C8 and C9. **Your program must use register level implementation. Directly calling a library (e.g. by using the Serial library written in C7) will not count!**

Write a sample code to test all your peripheral designs.

3.2 Control the DAC output

Using the code you've written in 3.1, write a simple program on your Il Matto that takes in a command from the **Serial Interface** and output the corresponding **PWM** wave.

Build a simple low-pass filter using the resistor and capacitor values worked out from your prep. Connect the PWM wave to the circuit. What did you observe? Try changing the PWM values and see what happens.

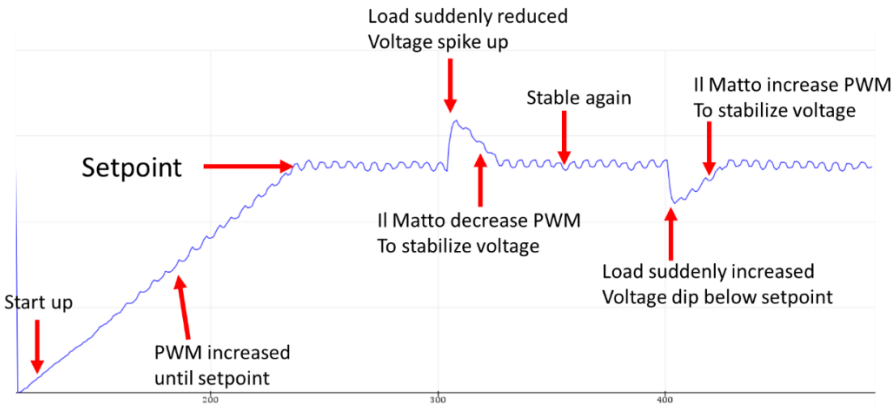
Observe the output and record your findings.

3.3 Implement ADC to read input

Using the code you have written for 3.1, **modify** your code on the Il Matto to read an analogue voltage value through your **ADC** class. You should be able to send this voltage value back to the serial monitor.

3.4 Implement a simple PID control system to maintain voltage levels

Implement a PID control system where the voltage levels are maintained even when small disturbance is created. To simulate this, you can measure the output voltage through a variable resistor. The output voltage should correct itself when you change the load resistor.



In your logbook, record the ripple voltages and the response time. Try to tune your system such that it has the best response time and least steady-state fluctuations.

© Electronics and Computer Science, University of Southampton

4 Optional Additional Work

⚠️ This section is an extension task for those who have already completed the main project. You will not get extra credits for this if your program doesn't work!

Tuning your controller takes time, and your Il Matto forgets everything at power-off. Use the AVR **EEPROM** library (<avr/eeprom.h>) to store your tuned values (Kp, Ki, Kd, setpoint) so the board powers up already tuned.

As a bonus, wrap this functionality as a **settings** class of its own, that can be used to change various settings on the firmware.

© Electronics and Computer Science, University of Southampton